Basics of Apache Spark

Recap: Why Hadoop & MapReduce?

Before understanding Spark, we revisited:

- Why the **Hadoop framework** was introduced → To store and process massive datasets distributed across machines.
- Why **MapReduce** was needed → To process big data in parallel using Map & Reduce phases.
- Role of **Resource Manager** → Negotiates and allocates resources for processing.
- Role of **HDFS** → Distributed storage for large-scale datasets.

Limitation Observed in MapReduce

- After every Map or Reduce step → Data is written back to **disk (HDFS)**.
- Intermediate results are repeatedly stored and read.
- If 200 operations are involved → 200+ disk writes/reads.
- Heavy **Disk I/O** → **High Latency** → **Slow Processing**.

This becomes a major bottleneck for:

- Iterative workloads
- Machine learning
- Interactive analytics

Why Apache Spark?

Apache Spark was introduced to overcome MapReduce limitations.

Key Improvements Over MapReduce

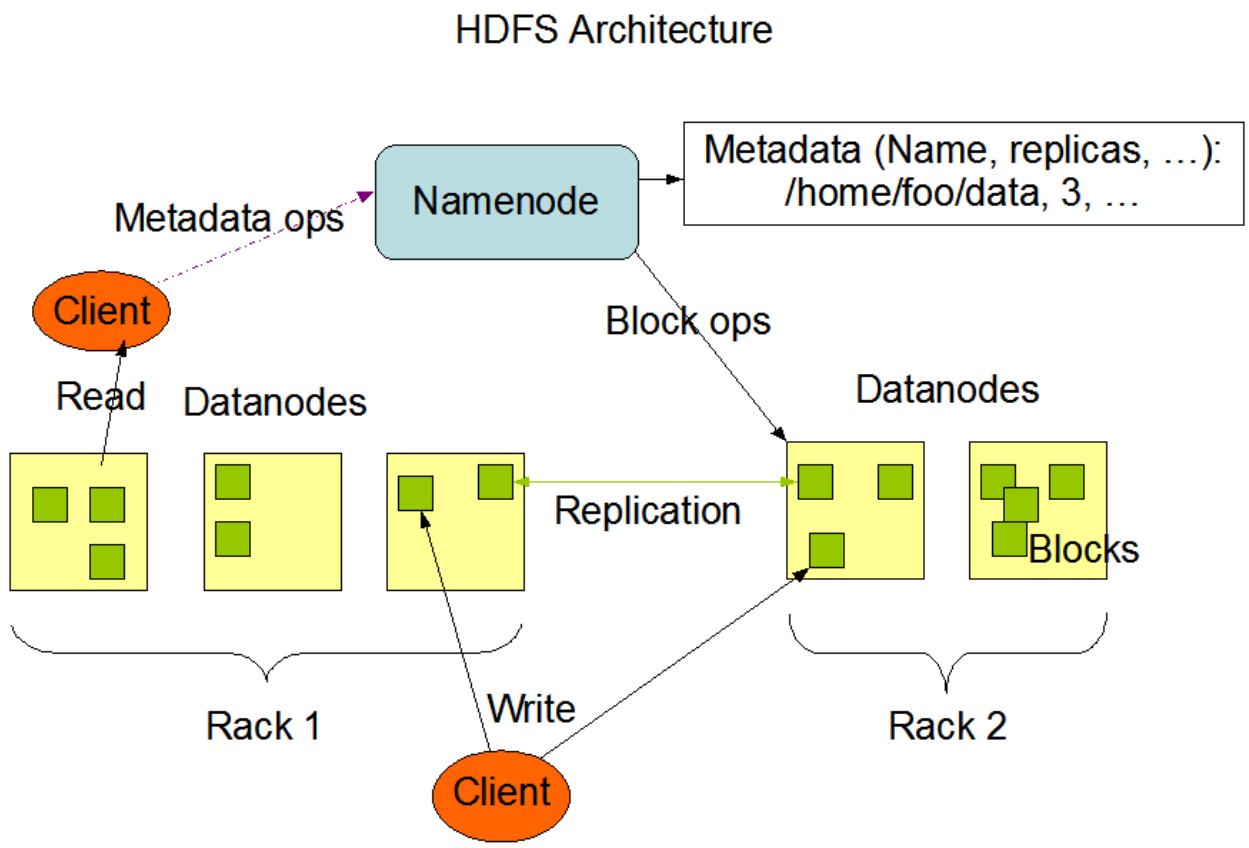
MapReduce	Spark
Disk-based processing	In-memory processing
High latency	Much faster execution
Complex programming	Simple APIs (Python, Scala, Java)
Batch-focused	Supports batch, streaming, ML, graph

Core Idea:

- Hadoop → Reliable storage (HDFS) + cluster management.
- Spark → Faster processing engine on top of Hadoop.

Spark:

- Can read from HDFS.
- Can use YARN for resource management.
- Does NOT replace Hadoop.
- Works on top of Hadoop ecosystem.



Classroom Analogy: MapReduce vs Spark

MapReduce (Repeated Collection & Storage)

Every step:

- Work given
- Collected
- Stored to disk
- Next work starts

Repeated writing causes delay.

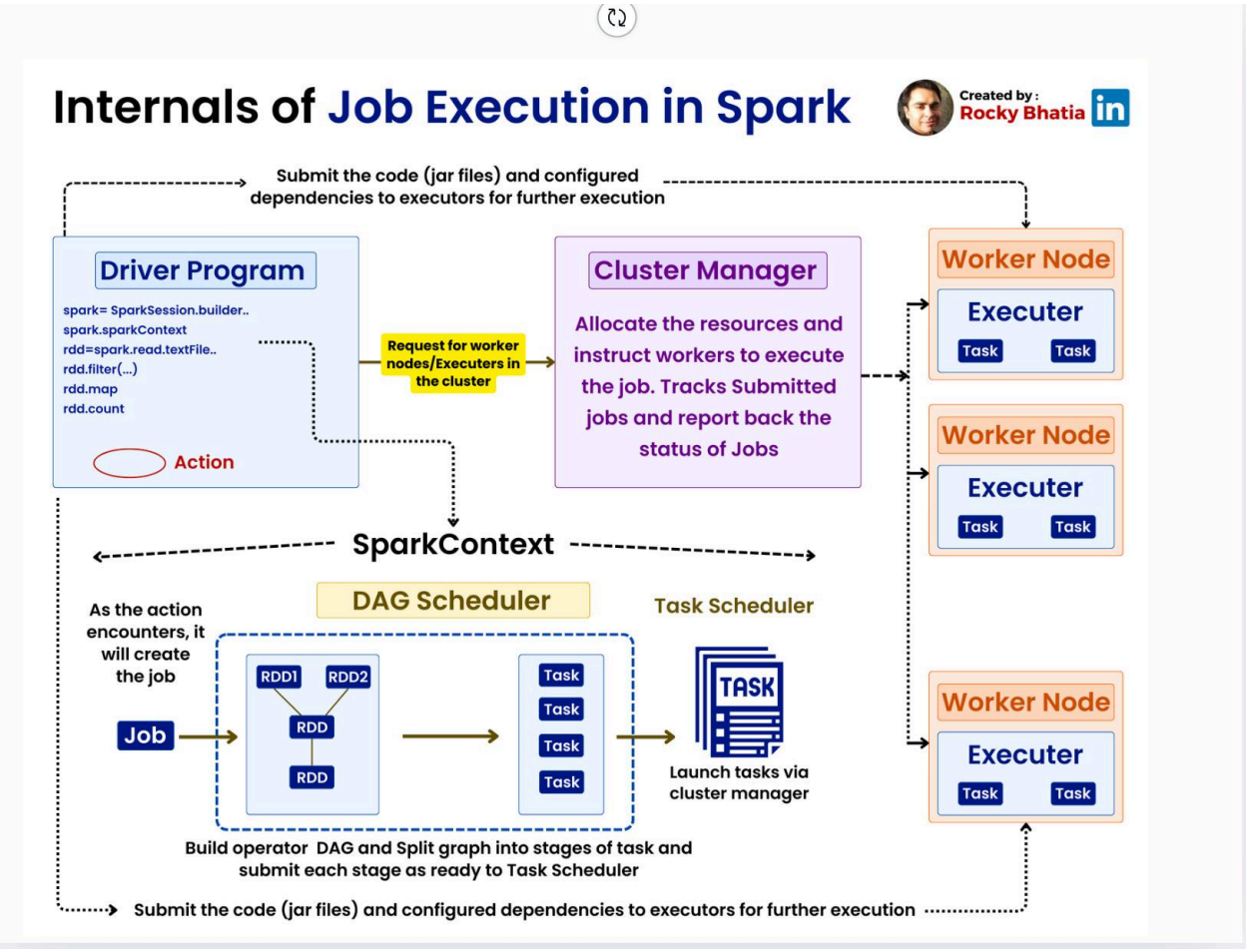
Spark

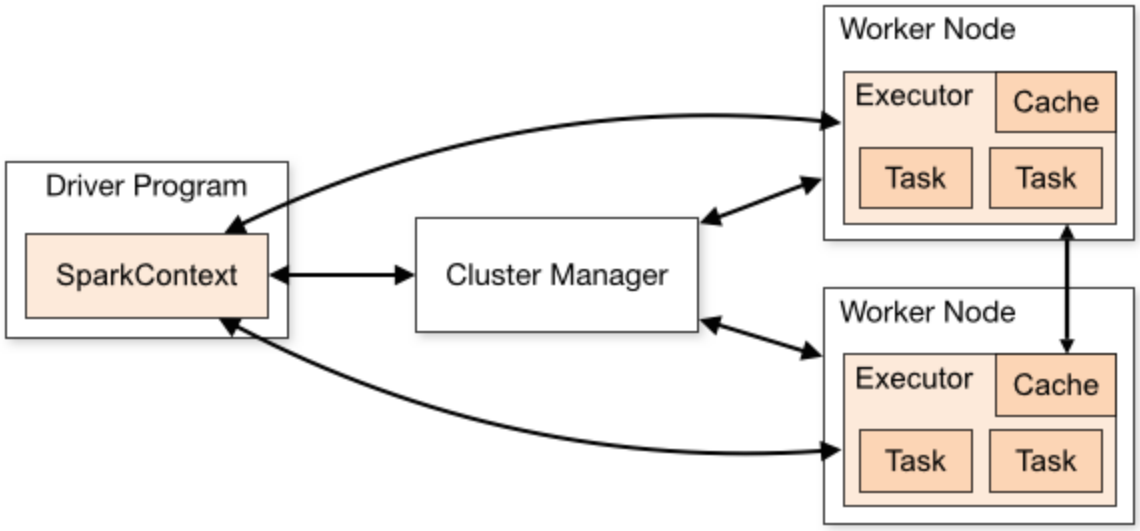
Every step:

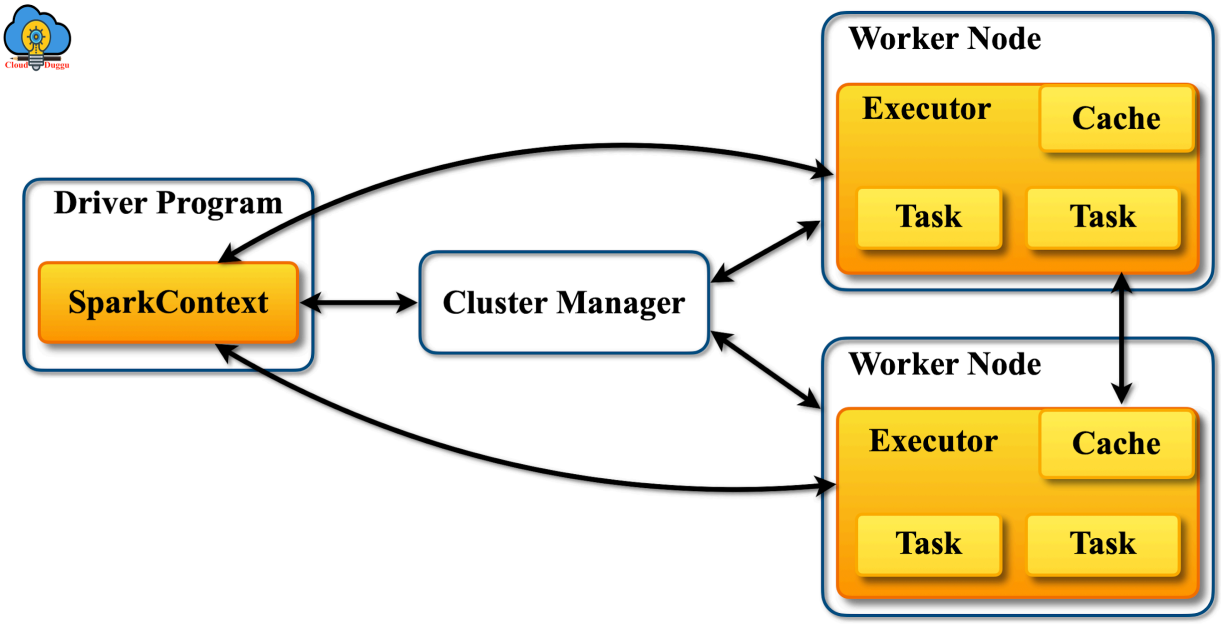
- Work stays in notebook
- Reused immediately
- No repeated disk writing
- Only final result written

Faster because data stays **in memory**.

Spark Architecture







Driver Program (Brain of Spark)

- Contains user code.
- Defines operations.
- Requests resources.

- Sends tasks to executors.
- Collects results.

Analogy: Teacher planning and managing class.

SparkContext (Entry Point)

- Gateway to Spark functionality.
- Connects driver to cluster.
- Used to create RDDs, DataFrames, Jobs.

Analogy: Teacher's microphone or notebook for instructions.

Cluster Manager

- Allocates CPU, memory, machines.
- Can be:
    - YARN
    - Mesos
    - Kubernetes
    - Standalone

Analogy: School Principal assigning classrooms.

Resource Manager

- Ensures resources are efficiently used.
- Balances workload.

Analogy: School administrator distributing desks & supplies.

Worker Node

- Machine where computation happens.
- Runs executors.

Analogy: Students in classroom.

Executor

- Process running on worker node.
- Executes tasks.
- Stores data in memory.
- Returns results to driver.

Analogy: Student's brain solving problems.

Task

- Smallest unit of work.
- Runs on a partition of data.

Analogy: Individual homework problem.

Cache

- Stores frequently used data in memory.
- Avoids recomputation.

Analogy: Student notebook for quick revision.

Spark Job Execution (Step-by-Step Flow)

1. Driver starts job.
2. SparkContext created.
3. Cluster Manager allocates resources.
4. Worker nodes ready.
5. Executors launched.
6. Job split into tasks.
7. Tasks executed in parallel.

8. Shuffle happens if required.
9. Results collected.
10. Resources released.

DAG (Directed Acyclic Graph)

What is DAG?

A **DAG** is Spark's execution plan.

Instead of executing line-by-line:

- Spark builds a logical plan.
- Breaks it into stages.
- Executes efficiently.

Components of DAG

1️⃣ Job

- Triggered by an **action** (count, collect, save).
- Top-level assignment.

2️⃣ Stage

- Logical grouping of tasks.
- Created based on shuffle boundaries.

3️⃣ Task

- Smallest execution unit.
- Handles one data partition.

Why "Acyclic"?

Acyclic = No loops.

Meaning:

- Execution always moves forward.
- No infinite repetition.
- Clear start and end.

Benefits:

- Predictable execution
- Fault tolerance
- Optimization possible
- Efficient recomputation

RDD (Resilient Distributed Dataset)

Definition

RDD is Spark's fundamental data structure.

It is:

- **Resilient** → Fault tolerant
- **Distributed** → Spread across cluster
- **Dataset** → Collection of records

Why RDD Was Introduced?

Problems with MapReduce

1. Disk-heavy processing
2. Slow iterative algorithms
3. Complex programming model

How RDD Solves These Problems

1️⃣ In-Memory Processing

- Data reused across steps.
- Much faster than disk-based systems.

2️⃣ Simple Programming Model

- High-level operations:
    - map
    - filter
    - reduce

3️⃣ Automatic Fault Recovery

- Uses **lineage**
- Recomputes only lost partition.
- No need to restart full job.

RDD Properties

- Immutable
- Partitioned
- Fault-tolerant
- Lazy evaluated
- Parallel processing

Where RDDs Are Highly Useful

1️⃣ Iterative Machine Learning

- Multiple passes over same dataset.
- Dataset stays in memory.
- Faster training.

2️⃣ Graph Processing

- Repeated traversal of nodes & edges.
- Efficient reuse of data.

3️⃣ Interactive Data Analysis

- Multiple exploratory queries.
- Cached data reused.
- Fast response.

Industry Use Cases

🛒 Online Retail Analytics

- Analyze product views, searches, purchases.
- Multiple transformations on same dataset.
- Faster exploratory analysis.

🎬 Recommendation Systems

- Iterative training.
- Repeated calculations.
- In-memory reuse improves performance.

🖥️ Log Analysis in IT Operations

- Large server logs.
- Filtering & aggregating repeatedly.
- Interactive troubleshooting.

🏦 Financial Risk & Fraud Detection

- Large historical datasets.
- Multiple recalculations.
- Faster risk modeling.
- Reliable fault recovery.

Key Interview Questions

🔹 Basic Level

- Why Spark is faster than MapReduce?
- What is DAG in Spark?
- What is RDD?
- Difference between Job, Stage, and Task?
- What happens when a node fails?

🔹 Intermediate Level

- Explain Spark architecture.
- What is lineage?
- What is shuffle?
- What is caching?
- How does Spark achieve fault tolerance?

🔹 Scenario Based

- Why is Spark suitable for ML?
- When would you still use MapReduce?
- How does Spark work with Hadoop?

Learner Practical Insights

- Spark does NOT replace Hadoop.
- Spark focuses on processing speed.

- Hadoop focuses on storage reliability.
- Spark is preferred for:
    - Machine Learning
    - Real-time analytics
    - Interactive workloads
- MapReduce is still useful for:
    - Large-scale batch jobs with simple transformations.

Final Recap

✔ Hadoop solved big data storage and distributed processing.
✔ MapReduce enabled parallel computation but suffered from disk latency.
✔ Spark introduced in-memory processing for faster execution.
✔ DAG ensures structured, optimized execution.
✔ RDD is Spark's core resilient distributed data structure.
✔ Spark supports modern workloads like ML, graph processing, and interactive analytics.

Career Insight

Apache Spark became the **next major evolution in the Big Data ecosystem** because:

- It reduces latency.
- It simplifies programming.
- It supports modern analytics workloads.
- It integrates with Hadoop ecosystem.
- It is widely used in industry.